

TESTAUTOMATION OVERVIEW

Presented by: Sandro Vincenti

Neu-Isenburg, 18.06.2026



CONTENTS

1. **MOTIVATION**
2. **TECHNOLOGY USED FOR TEST AUTOMATION**
3. **TEST AUTOMATION ARCHITECTURE**
 - a. **OVERVIEW**
 - b. **TREX**
 - c. **GITLAB PIPELINES**
4. **ROBOT FRAMEWORK**
 - a. **OVERVIEW**
 - b. **WHY ROBOT FRAMEWORK?**
 - c. **TEST RESULTS**
 - d. **IN DEPTH LOOK AT A LOG-FILE**
5. **USE CASE: LONG TERM TRAFFIC TEST**

1. MOTIVATION

Why do we need Test Automation?



Reproducibility & Consistency

- Manual tests are hard to reproduce
- Automation ensures (near) identical test conditions every time



Realistic & Long-Running Traffic

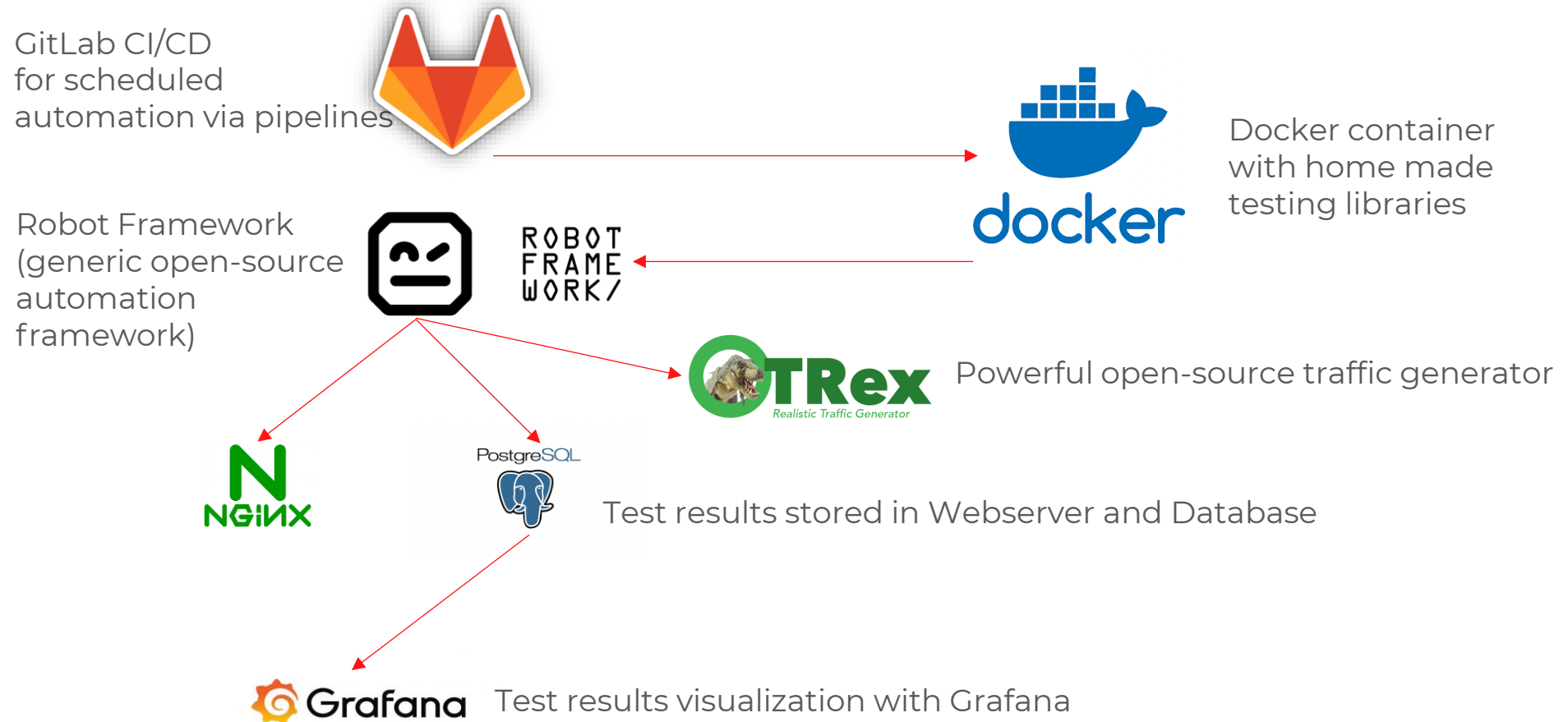
- Many issues appear only under sustained and realistic load
- Can test scenarios manual testing can't simulate



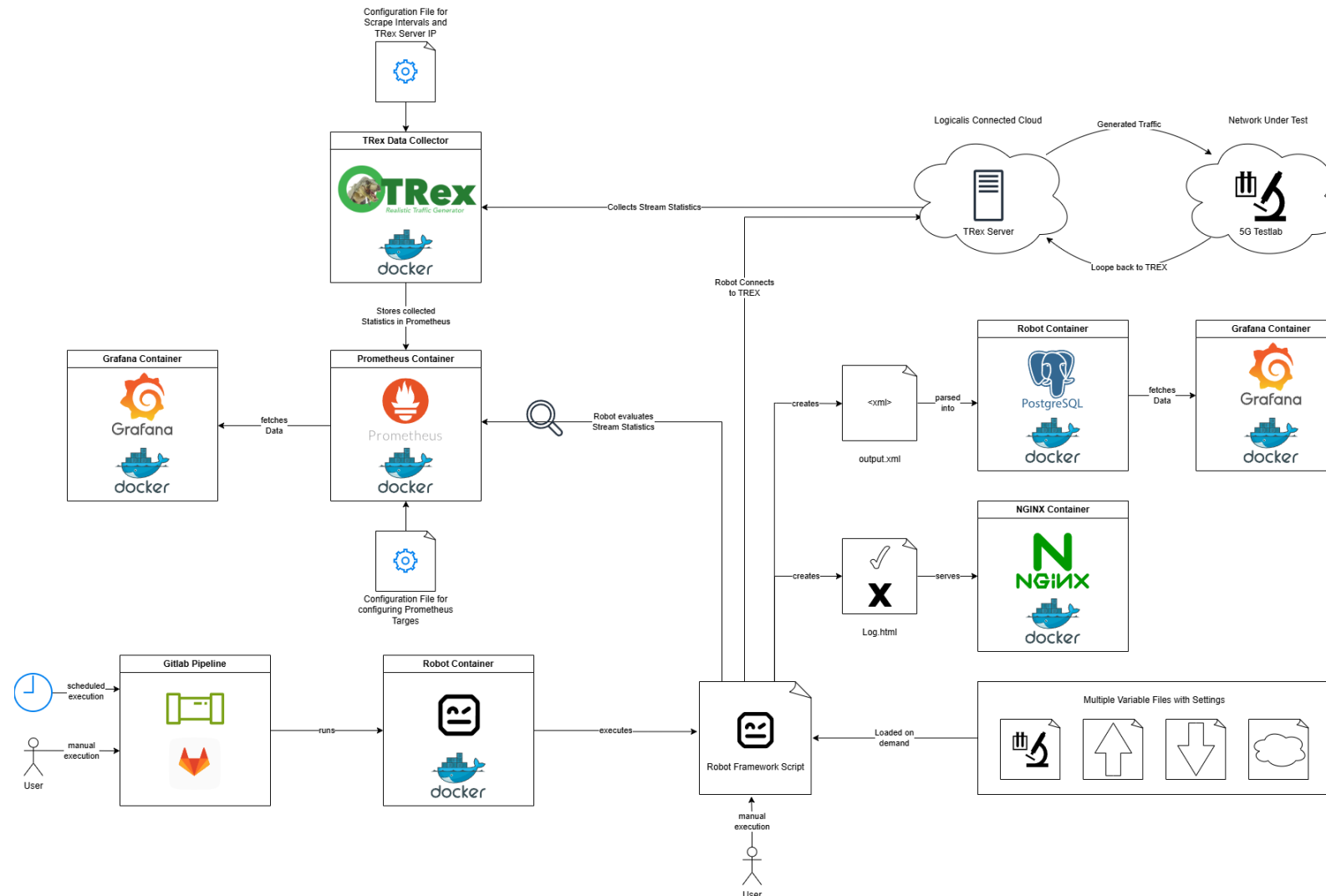
Scalability & Efficiency

- Engineers focus on analysis instead of execution
- Manual testing does not scale with system complexity

2. TECHNOLOGY USED



3. ARCHITECTURE – OVERVIEW



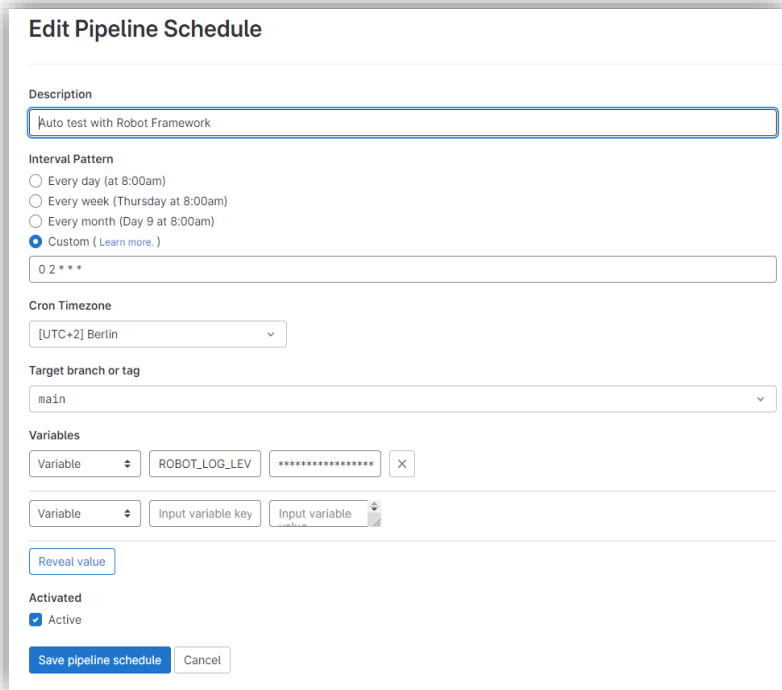
3. ARCHITECTURE – TREX

- ▶ Originally developed in 2003 by Cisco
- ▶ Open-source, highly performative and not vendor-locked
- ▶ Stateless and stateful traffic modes
- ▶ Can scale up to 200Gb/sec with one server
- ▶ Packets are created via Scapy and can therefore be built depending on the needs



3. ARCHITECTURE – GITLAB PIPELINES

Scheduled periodic tests
execution



Edit Pipeline Schedule

Description

Interval Pattern

☐ Every day (at 8:00am)

☐ Every week (Thursday at 8:00am)

☐ Every month (Day 9 at 8:00am)

☒ Custom ([Learn more.](#))

Cron Timezone

Target branch or tag

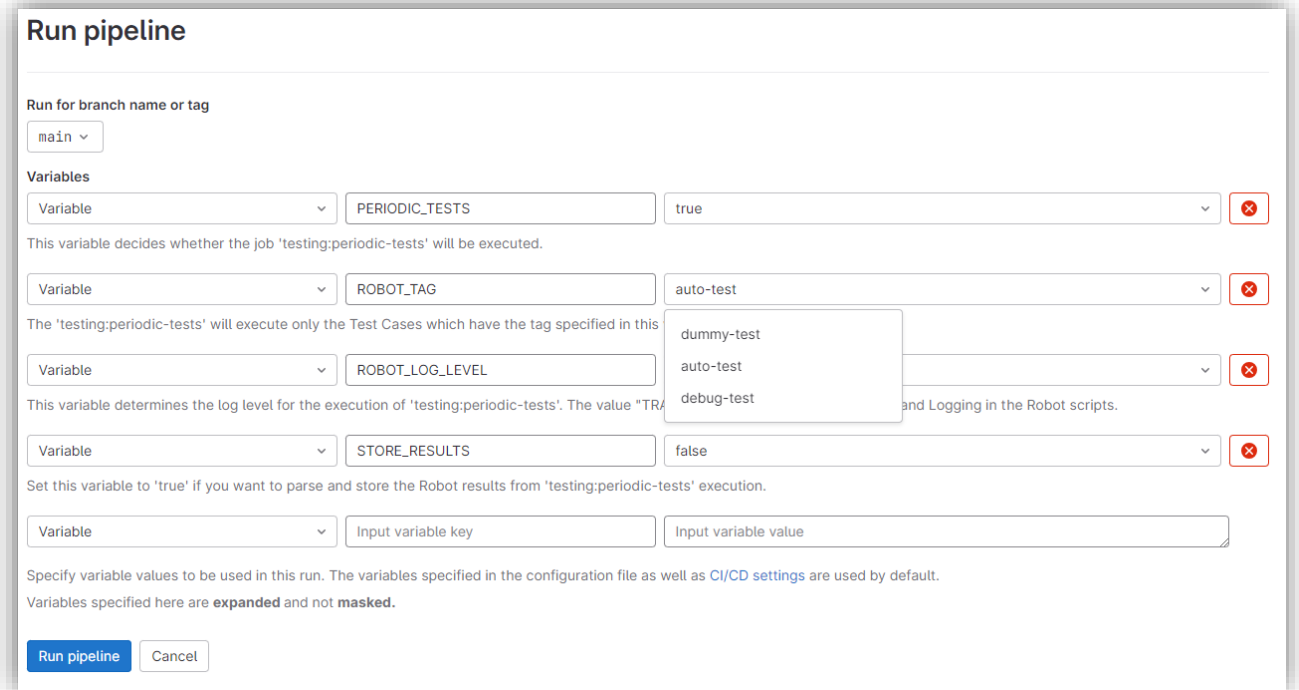
Variables

Active

☒ Active

[Save pipeline schedule](#) [Cancel](#)

Test execution could be triggered manually
with custom parameters



Run pipeline

Run for branch name or tag

Variables

This variable decides whether the job 'testing:periodic-tests' will be executed.

This 'testing:periodic-tests' will execute only the Test Cases which have the tag specified in this

This variable determines the log level for the execution of 'testing:periodic-tests'. The value "TR" and Logging in the Robot scripts.

Set this variable to 'true' if you want to parse and store the Robot results from 'testing:periodic-tests' execution.

Specify variable values to be used in this run. The variables specified in the configuration file as well as [CI/CD settings](#) are used by default. Variables specified here are **expanded** and not **masked**.

[Run pipeline](#) [Cancel](#)



4a. ROBOT FRAMEWORK – OVERVIEW

Robot Log-File

Test case built out of keywords in natural language

```
*** Variables ***
${UE_Identifier}=    Robustel_03

*** Test Cases ***
[TC_ID_2.1.1.]: Registration and PDU Session Establishment
[Tags]    end-to-end    device:${UE_Identifier}
BuiltIn.Log    \nUE in use is "${UE_Identifier}"    console=${True}
Load Device Information From File
...    filepath=devices.yaml
UE Setup
...    identifier=${UE_Identifier}
...    power_state=${True}
...    keep_connection_alive=${True}
...    register_core=${True}
...    pdu_session=${True}
UE Ping    identifier=${UE_Identifier}    ip_adress=8.8.8.8    repetitions=10    average_upper_limit=100
```

CASPA Periodic Test Log

Generated: 20230714 02:52:23 (UTC+02:00)
4 days, 12 hours ago

Test Statistics

Total Statistics		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests		5	5	0	0	00:49:12	

Statistics by Tag

Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
auto-test	5	5	0	0	00:49:12	

Statistics by Suite

Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
CASPA Periodic Test	5	5	0	0	00:49:12	
caspa-ue Multiple Registrations and Deregistrations-CAS-379-Stability-Regression	1	1	0	0	00:07:59	
caspa-ue Ping-Period greater than U-plane-inactivity timer-CAS-364-Stability-Regression	1	1	0	0	00:11:26	
caspa-ue Ping-Period less or equal U-plane-inactivity timer-CAS-364-Stability-Regression	1	1	0	0	00:10:48	
caspa-ue Registration and PDU Session Establishment-CAS-358-End-To-End-Regression	1	1	0	0	00:07:25	
caspa-ue Registration and PDU Session Establishment multiple use-CAS-359-End-To-End-Regression	1	1	0	0	00:11:32	

Test Execution Log

CASPA Periodic Test

Full Name: CASPA Periodic Test

Comment: main

Comment: x39f1e0951a1db972c308094c17b2d219076

Pipeline ID: 18544

Pipeline Source: caspa-test-automation

Project Name: caspa-test-automation

Source: build-infra/caspa-test-automation/test-suite

Start / End / Elapsed: 20230714 02:03:11.513 / 20230714 02:52:23.618 / 00:49:12.105

Status: 5 tests total, 5 passed, 0 failed, 0 skipped

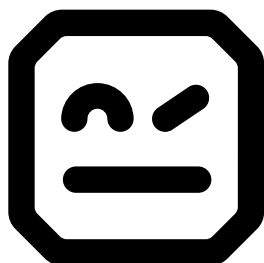
- Multiple Registrations and Deregistrations-CAS-379-Stability-Regression
- Ping-Period greater than U-plane-inactivity timer-CAS-364-Stability-Regression
- Ping-Period less or equal U-plane-inactivity timer-CAS-364-Stability-Regression
- Registration and PDU Session Establishment-CAS-358-End-To-End-Regression
- Registration and PDU Session Establishment multiple use-CAS-359-End-To-End-Regression

XML output – used for postprocessing

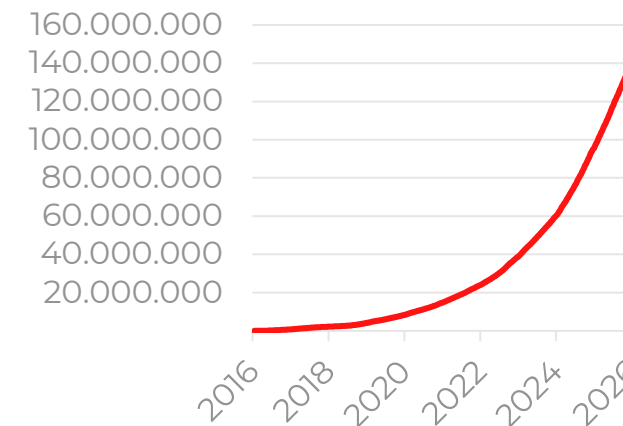
```
<?xml version="1.0" encoding="UTF-8"?>
<robot generator="Robot 6.0 (Python 3.10.12 on linux)" generated="20230713 08:31:35.136" rpa="false">
<suite id="s1" name="Multiple-Registrations-and-Deregistrations-CAS-379-Stability-Regression" source="<doc>Read and Validate the contents of the system configuration file</doc>
<kw name="Set Up Test Environment" type="SETUP">
<tag>auto-test</tag>
<tag>debug-test</tag>
<kw name="Setup System Configuration Environment" library="SystemConfigurationLibrary">
<doc>Read and Validate the contents of the system configuration file</doc>
<kw name="Read System Configuration File" library="SystemConfigurationLibrary">
<arg>${system_configuration_file}</arg>
<doc>Read and Scan System Configuration File</doc>
<msg timestamp="20230713 08:31:35.981" level="DEBUG">System Configuration read -&gt; {'system-confi
<status status="PASS" starttime="20230713 08:31:35.888" endtime="20230713 08:31:35.981"/>
</kw>
<kw name="Validate System Configuration" library="SystemConfigurationLibrary">
<doc>Validate System Configuration structure and consistency</doc>
<status status="PASS" starttime="20230713 08:31:35.982" endtime="20230713 08:31:35.983"/>
</kw>
<status status="PASS" starttime="20230713 08:31:35.882" endtime="20230713 08:31:35.983"/>
</kw>
```


4b. ROBOT FRAMEWORK – WHY RBF?

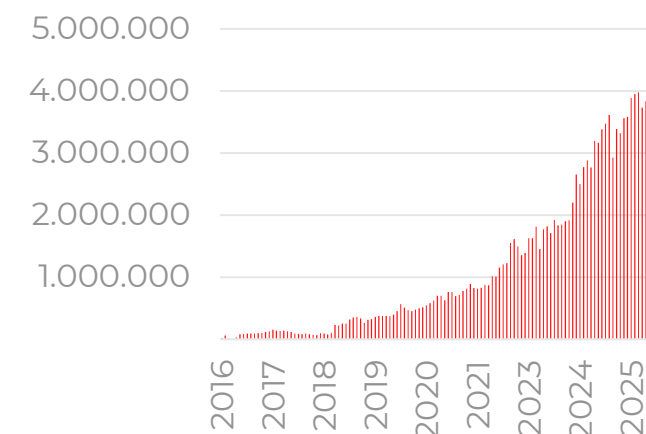
- ▶ Originally developed 2008 at Nokia
- ▶ Easy to write and understand
- ▶ Easy to implement in an existing Python environment
- ▶ Test Logs can be configured to provide information depending on user
- ▶ Once keywords have been provided new test cases are easy to write
- ▶ Written in Python and highly extensible



PyPI cumulative downloads



PyPI monthly downloads



4c. ROBOT FRAMEWORK – TEST RESULTS

The robot log files can be viewed at a web server

The output.xml file is processed, and the results are stored in a database

This database is used to visualize the results using Grafana

CASPA Periodic Test Log

Generated: 20230711 02:46:24 UTC+02:00
7 days 12 hours ago

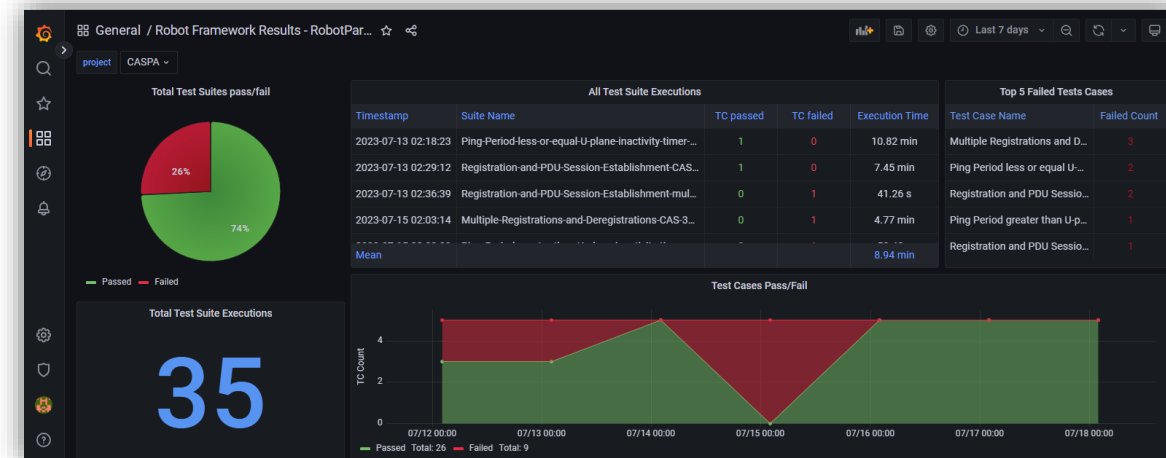
REPORT
Log level: INFO

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	5	2	3	0	00:43:11	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
auto-test	5	2	3	0	00:43:11	

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
CASPA Periodic Test	5	2	3	0	00:43:11	
CASPA Periodic Test: Multiple-Registrations-and-Deregistrations-CAS-379-Stability-Regression	1	1	0	0	00:19:23	
CASPA Periodic Test: Ping-Period-greater-than-U-plane-inactivity-timer-CAS-365-Stability-Regression	1	0	1	0	00:01:16	
CASPA Periodic Test: Ping-Period-less-or-equal-U-plane-inactivity-timer-CAS-364-Stability-Regression	1	0	1	0	00:00:47	
CASPA Periodic Test: Registration-and-PDU-Session-Establishment-CAS-358-End-To-End-Regression	1	1	0	0	00:07:24	
CASPA Periodic Test: Registration-and-PDU-Session-Establishment-multiple-use-CAS-359-End-To-End-Regression	1	0	1	0	00:14:21	



4d. ROBOT FRAMEWORK – LOG FILES

Test Catalogue Log

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	6	3	3	0	00:06:56	
Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
end-to-end	3	1	2	0	00:01:35	
performance	1	0	1	0	00:03:26	
Robustel_03	3	1	2	0	00:01:35	
Robustel_04	5	2	3	0	00:06:38	
stability	2	2	0	0	00:01:55	
Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Test Catalogue	6	3	3	0	00:06:57	
Test Catalogue . TC ID 2.1.1	1	1	0	0	00:00:19	
Test Catalogue . TC ID 2.1.2	1	0	1	0	00:00:21	
Test Catalogue . TC ID 2.1.2a	1	0	1	0	00:00:56	
Test Catalogue . TC ID 6.1.1a	1	0	1	0	00:03:26	
Test Catalogue . TC ID 7.1.1	1	1	0	0	00:00:57	
Test Catalogue . TC ID 7.1.2	1	1	0	0	00:00:57	

Test Execution Errors

20240627 18:31:56.443	ERROR	Ping validation failed: packet loss of 100%
20240627 18:31:56.444	ERROR	Ping from the UE was not successful
20240627 18:32:30.674	ERROR	Ping validation failed: packet loss of 100%
20240627 18:32:30.674	ERROR	Ping from the UE was not successful
20240627 18:32:52.412	ERROR	Ping validation failed: packet loss of 100%
20240627 18:32:52.412	ERROR	Ping from the UE was not successful
20240627 18:32:59.100	ERROR	PCI should be (21, <class 'int'>) but was (17, <class 'int'>)
20240627 18:32:59.100	ERROR	RSRP was not between (-70, <class 'int'>) and (80, <class 'int'>) but was (-99, <class 'int'>)
20240627 18:32:59.100	ERROR	SINR was not between (13, <class 'int'>) and (20, <class 'int'>) but was (24, <class 'int'>)
20240627 18:36:18.189	ERROR	ERROR: Expected performance was not achieved!!!

Generated
20240627 18:38:13 UTC+02:00
1 year 215 days ago

1. Timestamp

2. Changeable Log Levels

3. Overview of Runtime and Pass/Fail

4. Statistics for each Tag

5. Statistics for each Suite

6. Overview of Errors

REPORT

Log level: INFO



4d. ROBOT FRAMEWORK – LOG FILES

Test Execution Log

SUITE

Test Catalogue

Full Name: Test Catalogue

Source: [/workspace/tests/test_catalogue](#)

Start / End / Elapsed: 20240627 18:31:16.296 / 20240627 18:38:12.892 / 00:06:56.596

Status: 6 tests total, 3 passed, 3 failed, 0 skipped

SUITE

TC ID 2.1.1

00:06:56.596

SUITE

TC ID 2.1.2

00:00:18.736

SUITE

TC ID 2.1.2

00:00:21.478

TEST

[TC_ID_2.1.2]: Registration and PDU Session Establishment multiple UEs

00:00:21.469

Full Name: Test Catalogue.TC ID 2.1.2[TC_ID_2.1.2]: Registration and PDU Session Establishment multiple UEs

Tags: end-to-end, Robustel_03, Robustel_04

Start / End / Elapsed: 20240627 18:31:35.056 / 20240627 18:31:56.525 / 00:00:21.469

Status: **FAIL**

Message: Ping from the UE was not successful

KEYWORD

Builtin: Set Log Level level=DEBUG

KEYWORD

Builtin: Log UE_1 in use is "\${UE_1_Identifier}", console=\${True}

KEYWORD

Builtin: Log UE_2 in use is "\${UE_1_Identifier}", console=\${True}

KEYWORD

TC_Library: Load Device Information From File filepath=devices.yaml

KEYWORD

TC_Library: UE Setup identifier=\${UE_1_Identifier}, power_state=\${True}, keep_connection_alive=\${True}, register_core=\${True}, pdu_session=\${True}

KEYWORD

TC_Library: UE Ping identifier=\${UE_1_Identifier}, ip_address=10.120.212.254, repetitions=10, average_upper_limit=100

KEYWORD

TC_Library: UE Setup identifier=\${UE_2_Identifier}, power_state=\${True}, keep_connection_alive=\${True}, register_core=\${True}, pdu_session=\${True}

KEYWORD

TC_Library: UE Ping identifier=\${UE_2_Identifier}, ip_address=10.120.212.254, repetitions=10, average_upper_limit=100

Documentation: Uses the UE_Ping method which every UE implements through the UeControlInterface. The Device needs to be initialized first.

Start / End / Elapsed: 20240627 18:31:53.798 / 20240627 18:31:56.525 / 00:00:02.736

18:31:53.798

INFO

Logging into '10.120.151.242:22' as 'admin'.

18:31:55.997

INFO

Read output: #

18:31:55.997

INFO

Executing command 'ping 10.120.212.254 repeat 10'.

18:31:56.432

INFO

Command exited with return code 1.

18:31:56.443

INFO

Ping output:

18:31:56.443

INFO

PING 10.120.212.254 (10.120.212.254): 56 data bytes

18:31:56.443

INFO

CHECKING CRITERIA - [FAIL] : PING SUCCESS RATE 0%

18:31:56.443

ERROR

Ping validation failed: packet loss of 100%

18:31:56.444

ERROR

Ping from the UE was not successful

18:31:56.444

FAIL

Ping from the UE was not successful

REPORT

Log level: INFO

00:00:00.001

00:00:00.000

00:00:00.000

00:00:00.004

00:00:03.487

00:00:11.804

00:00:03.436

00:00:02.736

1. „Failed“ Keywords Auto-Expand

2. Output showed depending on Log level

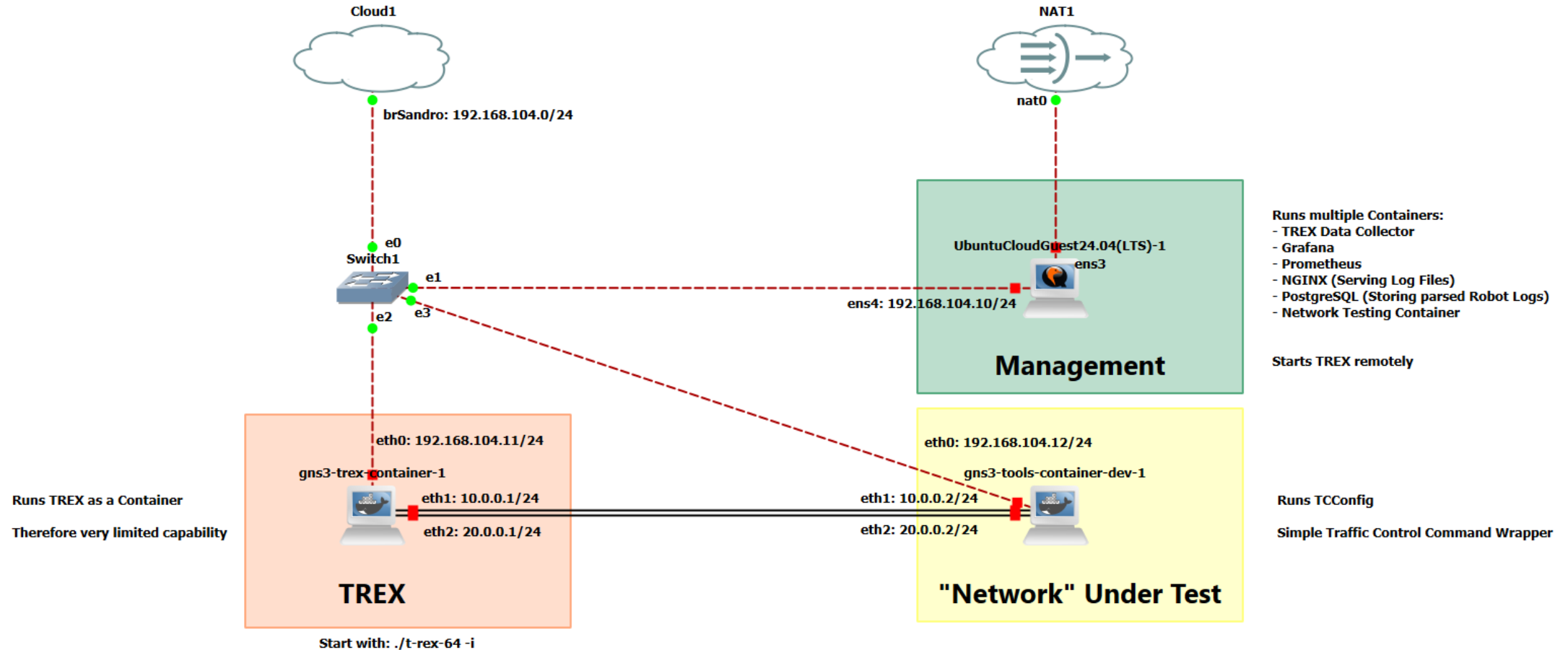


5. USE CASE: LONG TERM TRAFFIC TEST

For long term traffic tests, we use Prometheus and Grafana to store and visualize the traffic data



5. USE CASE: DEMO SETUP





5. USE CASE: DEMO

KEY TAKEAWAYS

- ▶ Manual testing doesn't scale but automation does!
- ▶ Robot Framework reduces automation complexity and builds naturally on existing (Python) Code.
(GO USE ROBOT!) 
- ▶ Traffic Generators don't have to be expensive to be valuable.

Thank You and Have a Great Day!

Let's stay connected!



de.logicalis.com



Join us on LinkedIn

CONTACT

Sandro Vincenti
sandro.vincenti@de.logicalis.com

BACKUP – LOG LEVEL

- ▶ Robot Log Level are adapted from Python's Logging Library
- ▶ Custom Log Levels may be added freely
- ▶ There already exist multiple Log Levels:

Log Level	Purpose	Internal Priority Value	
TRACE	Extremely detailed execution information	10	 More detailed More Critical
DEBUG	Debugging information	20	
INFO	Standard runtime information	30	
WARN	Warnings and unexpected behavior	40	
ERROR	Errors and failures	50	

BACKUP – TREX STATEFUL VS STATELESS



- ▶ In Short:
- ▶ **Stateful** = realistic application and flow simulation with session awareness
- ▶ **Stateless** = fast and flexible packet generation without connection context

Feature	Stateful	Stateless
Per flow state	Yes	No
NAT	Yes	No
Tunnel	Some are supported	Yes
L7 App emulation	Yes	No
Any type of packet	No	Yes
Latency Jitter	Global/Per flow	Per Stream

- ▶ “If you are confused, you probably need Stateless.” – TRex Documentation

BACKUP – TREX MINIMUM REQUIREMENTS



Resource	Minimum	Recommended*
CPU	2 dedicated x86 cores	8-16 physical cores
RAM	4 GB	16-32 GB
Storage	10 GB	-
NIC	1-2 NIC ports	Intel X710 / XL710 / XXV710 (generally DPDK supported NICs)
Expected Performance	~100 Mbps	Up to 200 Gbps

*https://trex-tgn.cisco.com/trex/doc/trex_book.pdf

BACKUP – TRAFFIC GENERATOR ALTERNATIVES

iPerf3	BNG Blaster	IXIA
Simple throughput tester	Broadband subscriber emulator	Enterprise traffic generator platform
No Vendor Lock	No Vendor Lock	Vendor Lock
Free	Free	\$\$\$\$
Network speed test for engineers	Specialized ISP testing tool	Professional testing appliance

BACKUP: ALTERNATIVE A – iPerf3

- ▶ Open-source network performance measurement tool
- ▶ TCP / UDP throughput testing
- ▶ Very easy to deploy
- ▶ Not a full traffic generator
- ▶ Limited protocol simulation
- ▶ A speed test between two servers
- ▶ Learn more: <https://iperf.fr/iperf-doc.php>

BACKUP: ALTERNATIVE B – BNG Blaster

- ▶ Specialized open-source ISP testing tool, developed by RtBrick
- ▶ Broadband subscriber emulation
- ▶ PPPoE, DHCP, BGP and routing protocols
- ▶ Designed for carrier and access networks
- ▶ Excellent for BNG validation
- ▶ A virtual ISP customer simulator.
- ▶ Learn more: <https://github.com/rtbrick/bngblaster>

BACKUP: ALTERNATIVE C – IXIA

- ▶ Enterprise-grade commercial traffic generator, now owned by Keysight
- ▶ Hardware appliances and software suites
- ▶ Large protocol library
- ▶ Industry standard in many labs
- ▶ High acquisition and licensing costs
- ▶ The professional test equipment used by vendors and large operators.
- ▶ Learn more: <https://ixia-c.dev/quick-start/introduction/>